Women's E-Commerce Clothing Review

Jessica, Carmen, Yanting, Emma, and Jakob

Business Problem & Objective

- The company receives thousands of customer reviews and cannot manually read every review. As a result, management struggles to quickly identify dissatisfied customers and recurring product issues related to sizing, fit, comfort, and product quality.
- Our business objective is to develop a sentiment analysis model that automatically classifies customer reviews as positive or negative so the company can identify dissatisfied customers more quickly and monitor common issues affecting customer satisfaction.

Data Overview

- Features:
  - Clothing ID, Age, Title, Review Text, Rating, Recommended IND, Posititve Feedback Count, Division Name, Department Name, and Class Name
- Amount of Observations:
  - There are 23,486 observations
- Data Types:
  - Mix of text, numerical, and categorical variables
- Source:
  - Real, anonymous customer reviews from a women's e-commerce retailer.
  - Gotten from Kaggle

Dropped All Unnecessary Columns & Missing Values

- We dropped all unnecessary columns, leaving only Review Text
- We also checked for missing values and found 859 of them

```
# These columns are not needed because our main goal is to
# analyze the review text and predict sentiment.

df = df.drop(columns=[
    "Unnamed: 0",
    "Clothing ID",
    "Title",
    "Recommended IND",
    "Positive Feedback Count",
    "Division Name",
    "Class Name"
])
```

```
print(df.isnull().sum())
```

```
Age                     0
Review Text           845
Rating                  0
Department Name        14
dtype: int64
```

```
df = df.dropna()
```

Duplicates

- We had 6 duplicate values and dropped them

```
# ============================================================
# STEP 7: CHECK FOR DUPLICATE REVIEWS
# ============================================================
# Duplicate reviews can bias the analysis, so we check and
# remove them.

print("Number of duplicate rows:", df.duplicated().sum())
```

```
Number of duplicate rows: 6
```

Tokenization and Standardizing Text

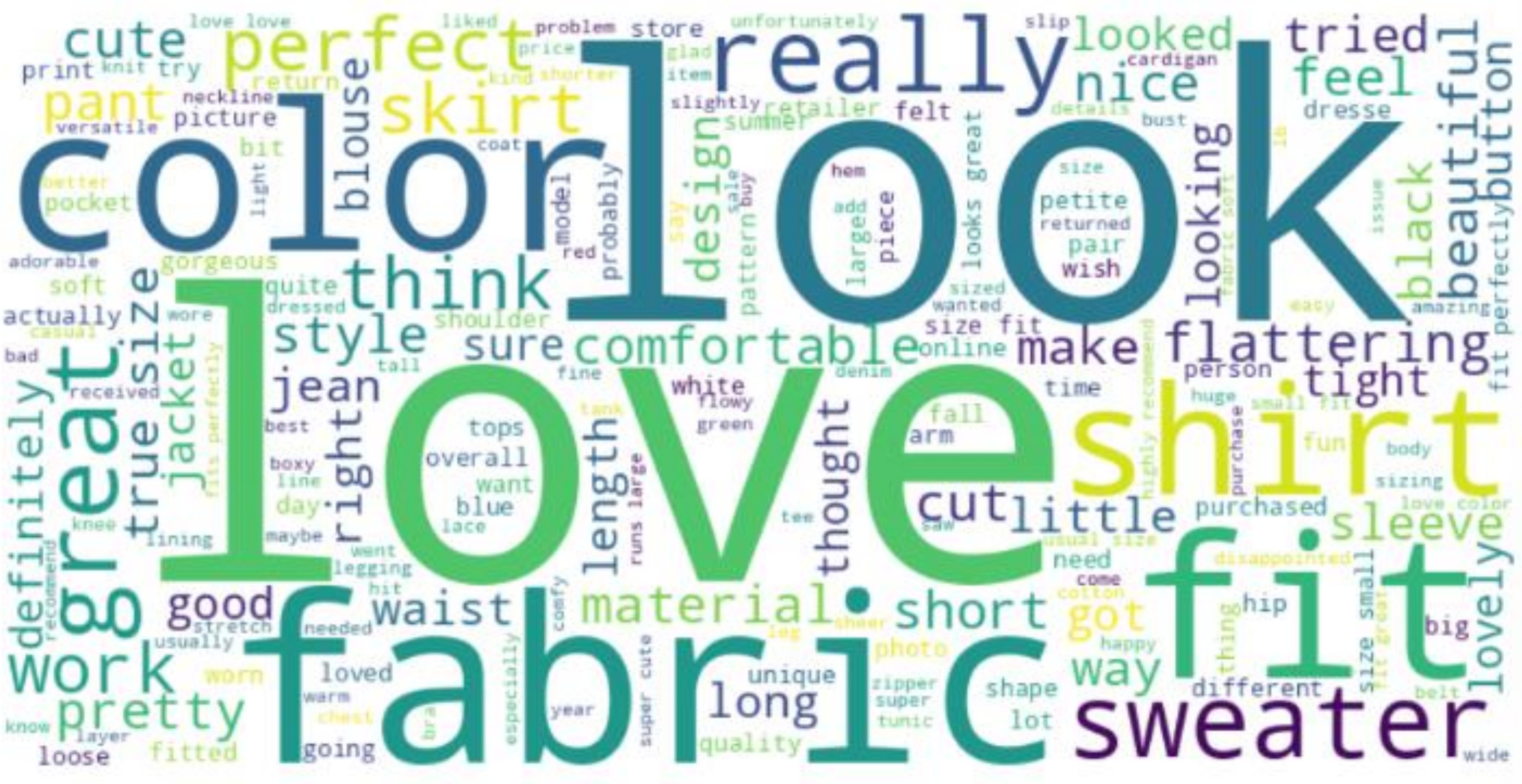
```
# ============================================================
# STEP 16: TOKENIZATION
# ============================================================
# Tokenization splits each review into individual words.
# This allows us to analyze text one word at a time.

from nltk import word_tokenize

df["tokens"] = df["Review Text"].apply(word_tokenize)

df[["Review Text", "tokens"]].head()


# ================================================
# STEP 18: LOWERCASE TEXT
# ================================================
# Convert all text to lowercase so that words such as
# Dress and dress are treated as the same word.

df["review_lower"] = df["Review Text"].str.lower()

df[["Review Text", "review_lower"]].head()
```

Keeping Alphabetic Tokens & Removing Stop Words

```
# ============================================================
# STEP 19: KEEP ALPHABETIC TOKENS
# ============================================================
# Remove punctuation and numbers.
# Keep only words made of letters.

df["alpha_tokens"] = df["tokens"].apply(
    lambda tokens:
    [word.lower() for word in tokens if word.isalpha()]
)

df[["tokens", "alpha_tokens"]].head()
```

```
# ============================================================
# STEP 21: REMOVE STOP WORDS
# ============================================================
# Stop words are very common words that usually do not add
# much meaning to sentiment analysis.

from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

custom_stopwords = ENGLISH_STOP_WORDS.union([
    "dress",
    "wear",
    "wearing",
    "ordered",
    "bought"
])

df["tokens_no_stopwords"] = df["alpha_tokens"].apply(
    lambda tokens:
    [word for word in tokens
     if word not in custom_stopwords]
)

df[["alpha_tokens", "tokens_no_stopwords"]].head()
```

Word Cloud



love
look
color
fabric
fit
shirt
sweater
really
great
perfect
skirt
cute
pant
think
comfortable
flattering
beautiful
button
tried
looked
nice
feel
black
looking
style
jean
material
short
waist
work
pretty
good
long
little
cut
size
true size
jacket
definitely
right
length
thought
design
blouse
sleeve
lovely
tight
make
sure
way
got
unique
loved
going
loose
fitted
quality
overall
tops
blue
want
purchased
different
petite
returned
pair
wish
dresse
person
time
print
try
picture
neckline
return
versatile
bit
pocket
adorable
soft
actually
gorgeous
quite
dressed
shoulder
store
item
slightly
retailer
felt
summer
bust
cardigan
details
price
glad
problem
liked
love love
unfortunately
slip
coat
light
better
hem
add
piece
large
looks great
model
probably
red
pattern
say
size
online
size fit
wanted
sized
fall
arm
white
flowy
green
fine
tall
denim
best
boxy
line
day
knee
lining
maybe
went
legging
hit
stretch
usually
needed
worn
warm
chest
know
layer
year
super cute
comfy
especially
runs large
tee
saw
need
come
cotton
photo
zipper
super
tunic
shape
lot
happy
thing
hip
big
size small
fit great
belt
wide
sizing
body
fun
love color
disappointed
usual size
purchase
highly recommend
huge
small fit
easy
amazing
issue
fit perfectly
wore
casual
bad
received
fits perfectly
recommend
hem
tank
sheer
leg
kind
shorter

Word Analysis

- Looked at the frequency of when words were mentioned

```
# ============================================================
# STEP 24: MOST COMMON WORDS
# ============================================================
# Identify the most frequently used words across all reviews.

all_words = []

for tokens in df["tokens_no_stopwords"]:
    all_words.extend(tokens)

top_words = pd.DataFrame(
    pd.Series(all_words).value_counts().head(20)
)

top_words.columns = ["frequency"]

top_words
```

	frequency
love	8931
size	8697
fit	7268
like	7006
great	6096
just	5596
fabric	4777
small	4561
color	4560
look	4018
really	3923
little	3771
perfect	3754
flattering	3485
did	3446
soft	3312
comfortable	3045
cute	3030
nice	3015
beautiful	2950

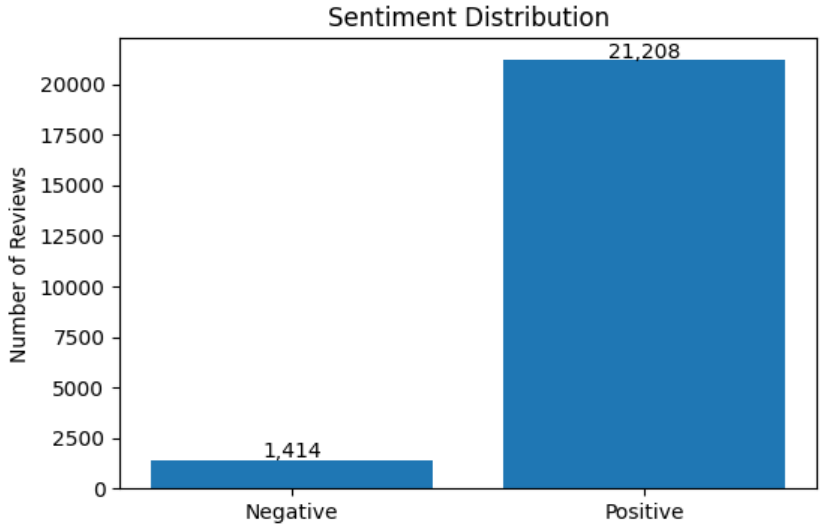
Polarity Before Sentiment

```
# ============================================================
# STEP 26: POLARITY ANALYSIS
# ============================================================
# Polarity measures how positive or negative a review is.
#
# Range:
# -1 = Very Negative
#  0 = Neutral
# +1 = Very Positive

!pip install textblob -q

from textblob import TextBlob

df["polarity"] = df["Review Text"].apply(
    lambda text: TextBlob(str(text)).sentiment.polarity
)
```

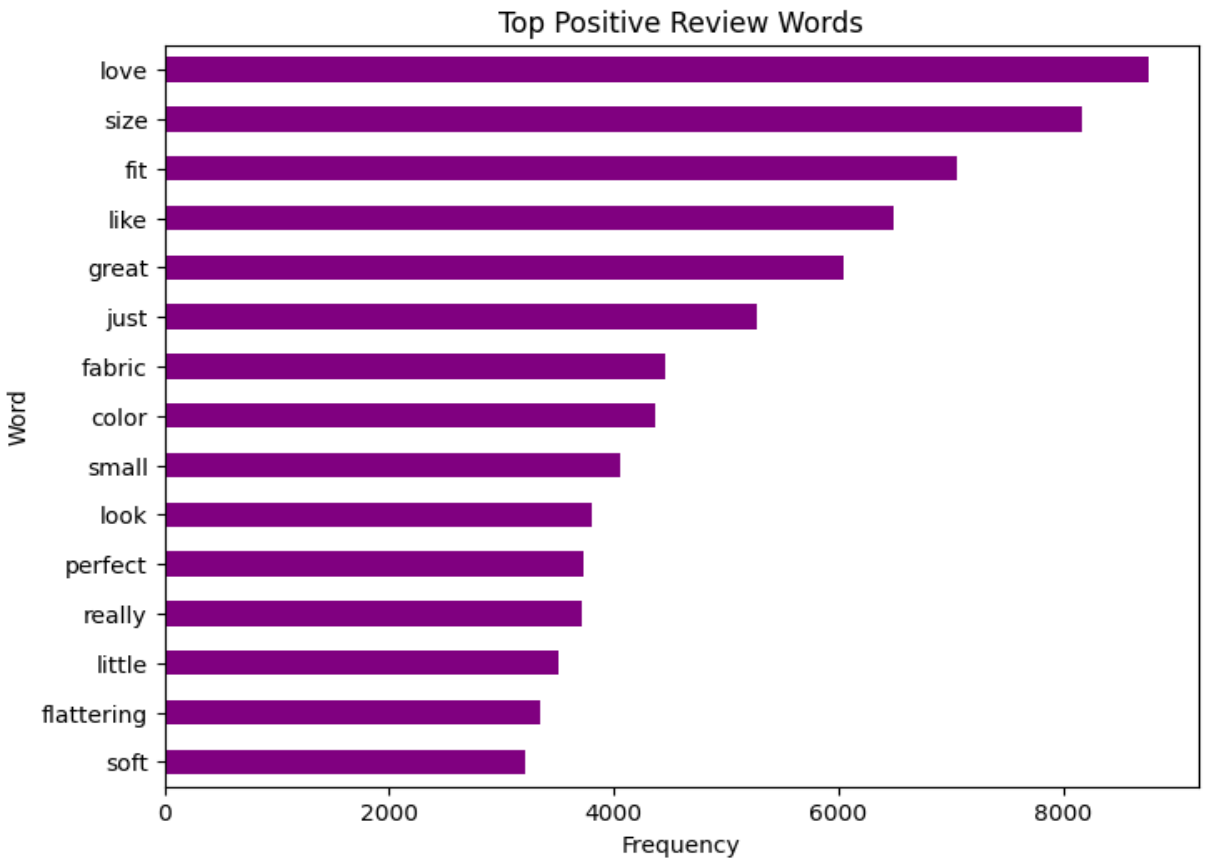
	Review Text	polarity
0	Absolutely wonderful - silky and sexy and comf...	0.633333
1	Love this dress! it's sooo pretty. i happene...	0.339583
2	I had such high hopes for this dress and reall...	0.073675
3	I love, love, love this jumpsuit. it's fun, fl...	0.550000
4	This shirt is very flattering to all due to th...	0.512891

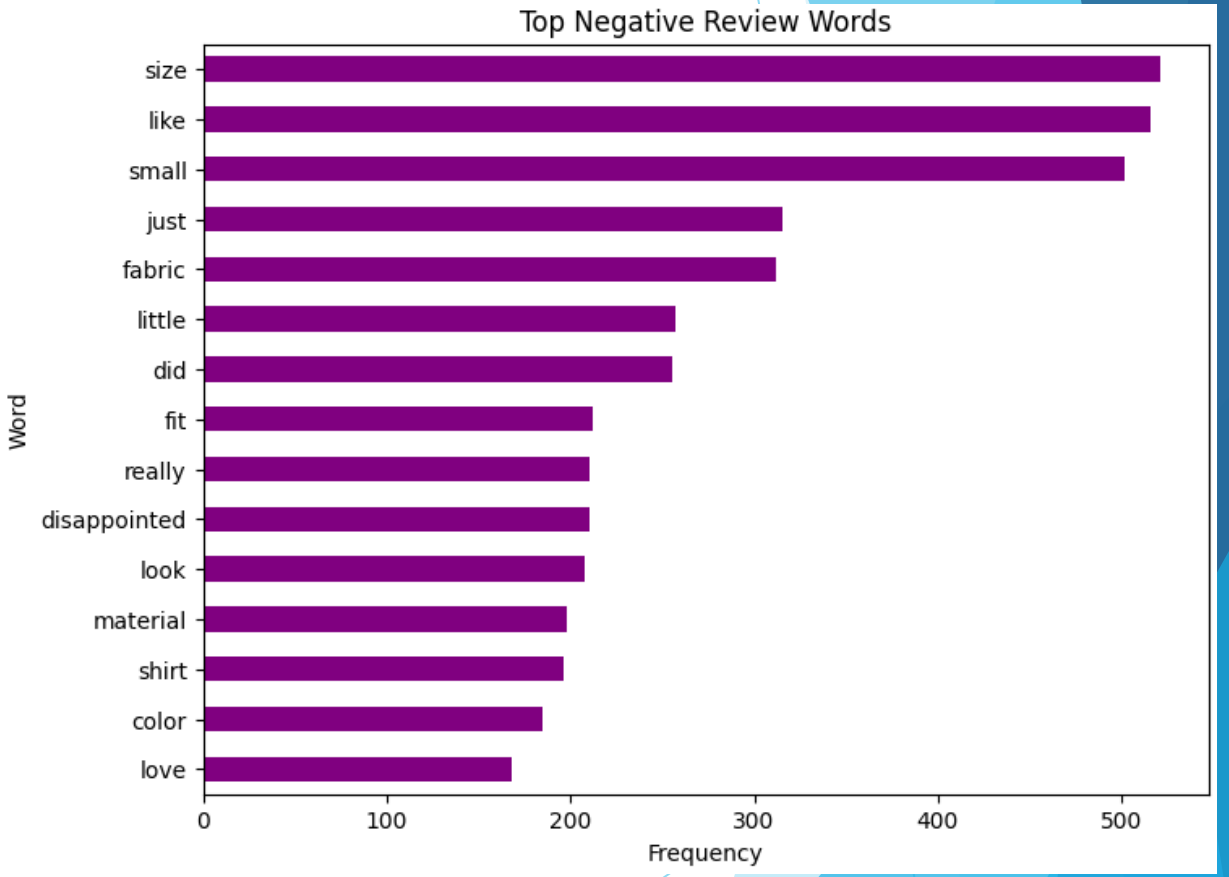
Sentiment Distribution

- Positive Reviews: 21,208
  - 93.75% positive
- Negative Reviews: 1,414
  - 6.25% Negative
- Dataset is highly imbalanced
- Used `class_weight="balanced"` to address the imbalance



Top Positive & Negative words





Bag-of-Words (BOW)

- Converts text into numerical features.
- Counts how often words appear.
- Vocabulary size: 20 words.

```
from sklearn.feature_extraction.text import CountVectorizer

count_vect = CountVectorizer(max_features=20)

X_bow = count_vect.fit_transform(
    df["clean_review_no_stopwords"]
)

bow_df = pd.DataFrame(
    X_bow.toarray(),
    columns=count_vect.get_feature_names_out()
)

bow_df.head()
```

	beautiful	color	comfortable	cute	did	fabric	fit	flattering	great	just	like	little	look	love	nice	perfect	really	size	small	soft
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	1	0	1	0	2	0	0	0	0	0	0
2	0	0	1	0	0	0	1	0	0	1	0	0	0	0	0	0	1	1	3	0
3	0	0	0	0	0	0	0	0	1	0	0	0	0	3	0	0	0	0	0	0
4	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0	0	0

TF-IDF

- TF-IDF
- Assigns importance weights to words.
- Common words receive lower weight.
- More meaningful words receive higher weight.

```
# ============================================================
# STEP 43: CREATE MODELING TF-IDF FEATURES
# ============================================================
# Fit TF-IDF on the training text only.
# Transform the test text using the same vocabulary.
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_model = TfidfVectorizer(
    max_features=300,
    stop_words="english",
    ngram_range=(1, 2),
    token_pattern=r"\b[^\d\W][^\d\W]+\b"
)

X_train = tfidf_model.fit_transform(X_train_text)

X_test = tfidf_model.transform(X_test_text)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
```

```
X_train shape: (18097, 300)
X_test shape: (4525, 300)
```

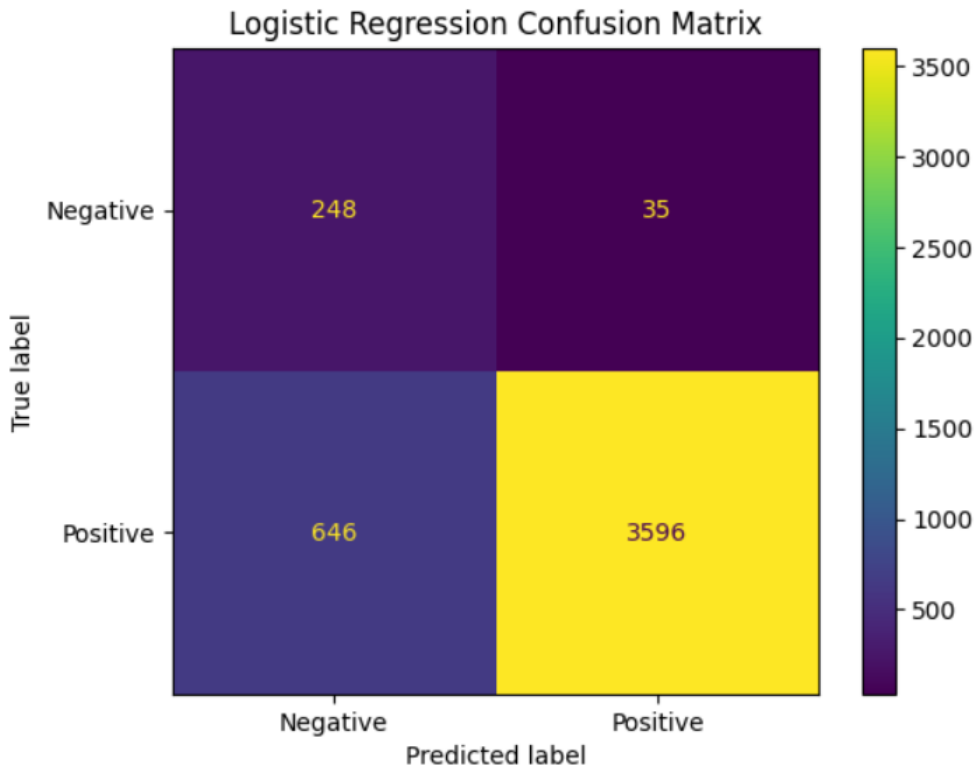
	beautiful	color	comfortable	cute	did	fabric	fit	flattering	great	just	like	little	look	love	nice	perfect	really	size	small	soft
0	0.0	0.0	1.000000	0.0	0.000000	0.0	0.000000	0.000000	0.000000	0.000000	0.0	0.000000	0.0	0.000000	0.0	0.000000	0.000000	0.00000	0.000000	0.0
1	0.0	0.0	0.000000	0.0	0.464918	0.0	0.000000	0.000000	0.000000	0.395147	0.0	0.450993	0.0	0.651395	0.0	0.000000	0.000000	0.00000	0.000000	0.0
2	0.0	0.0	0.294961	0.0	0.000000	0.0	0.223279	0.000000	0.000000	0.249085	0.0	0.000000	0.0	0.000000	0.0	0.000000	0.281406	0.21735	0.821383	0.0
3	0.0	0.0	0.000000	0.0	0.000000	0.0	0.000000	0.000000	0.363102	0.000000	0.0	0.000000	0.0	0.931749	0.0	0.000000	0.000000	0.00000	0.000000	0.0
4	0.0	0.0	0.000000	0.0	0.000000	0.0	0.000000	0.630787	0.000000	0.000000	0.0	0.000000	0.0	0.456913	0.0	0.627167	0.000000	0.00000	0.000000	0.0

Model Comparison (Traditional ML)

	Model	Accuracy	Negative Recall	Precision	Recall	F1 Score
0	Logistic Regression	0.85	0.88	0.99	0.85	0.91
1	Random Forest	0.94	0.13	0.95	1.00	0.97

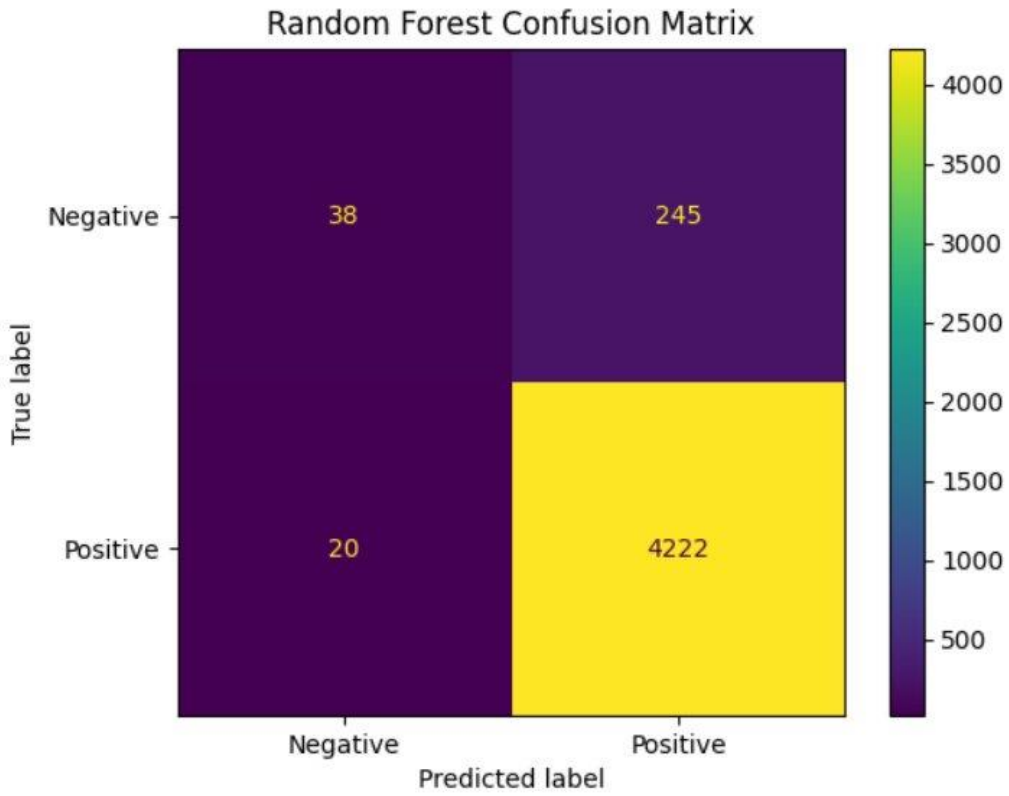
- Random Forest has higher accuracy
- Logistic Regression was selected as the best model because it identified substantially more dissatisfied customers.
- For business applications, identifying dissatisfied customers is often more important than maximizing overall accuracy.

Logistic Regression Results- Best Model



- Correctly classified 3,844 reviews:
  - 248 negative reviews
  - 3,596 positive reviews
- Incorrectly classified 681 reviews
  - 35 negative reviews were predicted as positive
  - 646 positive reviews were predicted as negative
- Successfully identified most dissatisfied customers, making it the preferred model for this business objective

Random Forest Results



- Correctly Classified 4,260 Reviews:
  - 38 negative reviews
  - 4,222 positive reviews
- Incorrectly classified 265 reviews:
  - 245 negative reviews were predicted as positive
  - 20 positive reviews were predicted as negative

Deep Learning Model (ANN)

- Architecture:

  Input Layer

  ↓

  Dense (32, ReLU)

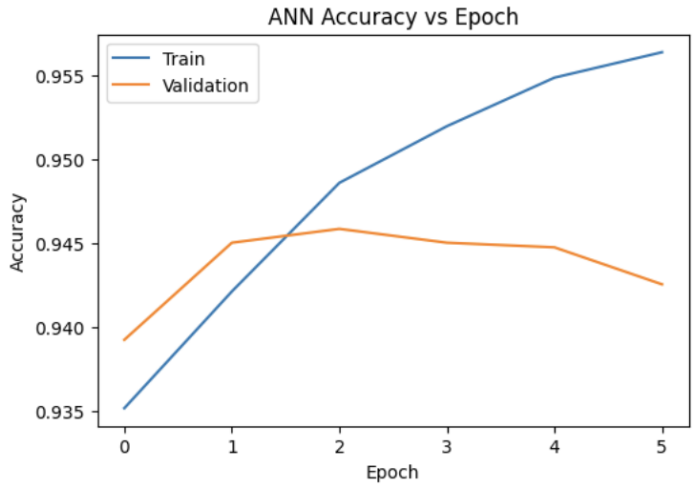
  ↓

  Dense (16, ReLU)

  ↓

  Output (Sigmoid)

- Epochs: 3
- Optimizer: Adam
- Loss Function: Binary Crossentropy
- Test Accuracy: 93.39%



- Validation accuracy peaked at Epoch 3.
- Additional training resulted in slight overfitting, so Epoch 3 was selected as the final ANN model.
- As an extension of our project, we also evaluated a Deep Learning approach using an Artificial Neural Network (ANN).

Final Model Comparison

	Model	Accuracy	Negative Recall	Precision	Recall	F1 Score
0	Logistic Regression	0.8495	0.8763	0.9904	0.8477	0.9135
1	Random Forest	0.9414	0.1343	0.9452	0.9953	0.9696
2	ANN	0.9339	0.3852	0.9595	0.9705	0.9650

- Random Forest achieved the highest accuracy.
- ANN demonstrated the potential of Deep Learning and achieved competitive performance.
- However, Logistic Regression achieved the highest Negative Recall and was most effective at identifying dissatisfied customers.
- Therefore, Logistic Regression was selected as the preferred model.

Recommendation 1

- Improve Product Sizing Guidance
- Findings:
  - "size" was the most common negative-review word
  - "small" and "fit" frequently appeared in dissatisfied customer reviews
- Recommendation:
  - Add product-specific sizing labels such as:
    - Runs small
    - Runs Large
    - True to Size
  - Display customer sizing feedback directly on product pages
  - Add measurement tables for each size & product

Recommendation 2

- Monitor fabric-related complaints
- Finding:
  - "Fabric" appeared frequently in both positive and negative reviews
  - Fabric quality strongly influences customer sentiment
- Recommendation:
  - Create a monthly dashboard tracking fabric-related complaints
  - Identify products with recurring comments about:
    - Thin fabric
    - Rough fabric
    - Poor quality material
  - Review those products with suppliers and merchandising teams

Conclusion

- We analyzed 23,486 customer reviews from a women's e-commerce retailer.
- Sentiment analysis showed that most reviews were positive, while sizing, fit, and fabric quality were the most common customer concerns.
- Although Random Forest achieved the highest accuracy and ANN demonstrated competitive performance, Logistic Regression was the most effective model for identifying dissatisfied customers.
- These findings can help businesses improve products, enhance customers satisfaction, and make more effective data-driven decisions.

Thank You !